# Mission-DT

Mission-Level Digital Twin for Hybrid Fleets of Physical and Virtual Unmanned Vehicles

## Application Manual

Version 1.0 — July 2026

LSA / PUCRS — github.com/ProfessorFilipo/mission-dt

# Contents

Contents ..... 2
1. Introduction ..... 3
1.1 Reading guide ..... 3
2. Architecture ..... 4
2.1 The frame loop ..... 4
2.2 MQTT contract ..... 4
2.3 Independence of components ..... 5
3. Installation ..... 6
3.1 Broker (Mosquitto) ..... 6
3.2 Python environment ..... 6
4. Running a mission ..... 6
4.1 3D view controls ..... 7
4.2 Status visualization ..... 7
4.3 The agent panel ..... 7
5. Configuration reference ..... 8
5.1 Route profiles ..... 8
5.2 Checkpoints in the 3D view ..... 8
6. Ready-to-run test scenarios ..... 9
Scenario T1 — Orbitas (default): swarm separation ..... 9
Scenario T2 — Paralelo: formation transit ..... 9
Scenario T3 — Explorador: aerial scouts ..... 9
Scenario T4 — Aleatorio: converging random routes ..... 9
Scenario T5 — Explicit shared checkpoints ..... 9
Scenario T6 — Communication loss ..... 9
Scenario T7 — Low battery ..... 9
Scenario T8 — Recording a video ..... 9
Scenario T9 — Reproducing the paper experiments ..... 10
Scenario T10 — Twin fidelity under packet loss (E4) ..... 10
7. Measured behaviour (summary of experiments) ..... 11
8. Troubleshooting ..... 13
9. Connecting physical vehicles ..... 13

# 1. Introduction

Mission-DT is a mission-level digital twin (DT) for fleets of unmanned vehicles. Unlike vehicle- or fleet-level twins, Mission-DT promotes the mission itself to the twinned entity: a single real-time model that tracks the state of every agent, decomposes mission goals into per-agent goals, computes the mission context (such as inter-agent distances), and issues actuation — once every 125 ms frame.

Its defining behaviour is hybridity. Each agent in the fleet is either a physical vehicle (an ArduPilot autopilot bridged into MQTT) or a virtual agent — itself an independent digital twin that emulates a vehicle, with its own kinematic model, its own state, and its own MQTT connection. The mission core cannot tell the two apart: both honour the same telemetry and actuation topics. This enables mixed-reality operation (for example, rehearsing a twenty-vehicle mission with three real vessels and seventeen virtual ones), incremental fleet deployment, and safe what-if analysis.

The stack is deliberately lightweight. The core is plain Python over a Mosquitto MQTT broker; the 3D mission view is a decoupled game-engine client (Ursina/Panda3D); a live agent panel runs in any terminal. The whole system meets its 125 ms real-time frame deadline for fleets of up to 100 agents on a single virtual CPU, and has been validated on Linux (x86), Windows 10 (x86), and macOS (Apple Silicon).

## 1.1 Reading guide

- Section 2 explains the architecture and the MQTT contract.
- Section 3 covers installation on Windows, macOS, and Linux.
- Section 4 shows how to run a mission and use the 3D view and the panel.
- Section 5 is the configuration reference (fleets, profiles, checkpoints).
- Section 6 provides ready-to-run test scenarios for every usage profile.
- Section 7 summarizes the measured behaviour (experiments E1–E3).
- Section 8 covers troubleshooting, drawn from real deployment issues.

# 2. Architecture

Figure 1 shows the three groups of components. The ground station hosts a local MQTT broker, the Mission-DT core, and the 3D mission view. A physical agent runs the ArduPilot firmware and a thin MAVLink-to-MQTT bridge behind its own local broker, connected to the ground station in bridge mode over Wi-Fi. A virtual agent is a self-contained vehicle twin: it integrates a kinematic model and samples synthetic sensors at 50 Hz, publishing through a bandwidth regulator that decimates the stream to the 8 Hz frame rate.

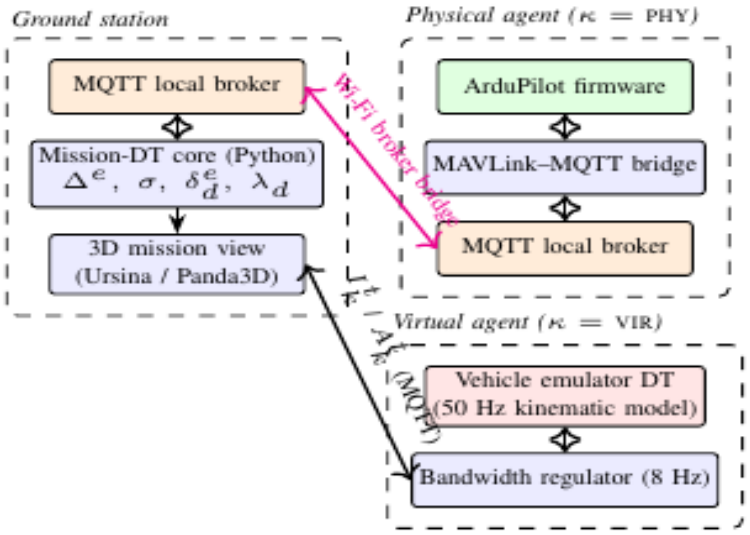


*Figure 1 — Mission-DT architecture. Physical and virtual agents share identical MQTT topics.*

## 2.1 The frame loop

The core subscribes to every agent's telemetry and runs a fixed-period loop (125 ms). Each frame it: (1) consumes at most the newest pending telemetry message per agent, discarding outdated packets by sequence number; (2) updates the agent state, dead-reckoning from the state history when telemetry misses a frame (only after the first telemetry has seeded the state); (3) evaluates the mission context — pairwise inter-agent distances — and a separation rule that overrides the goal-seeking controller with corrective actuation when two same-domain agents come closer than 12 m; and (4) publishes actuation. The colour behaviour of the safety spheres in the 3D view mirrors these thresholds exactly.

## 2.2 MQTT contract

| Topic | Retained | Purpose |
|---|---|---|
| missiondt/agents/<id>/telemetry | no | Agent → core: sensor readings (GPS, attitude, velocity, IMU, battery), ~560-byte JSON at 8 Hz |
| missiondt/agents/<id>/actuation | no | Core → agent: throttle, steering, climb; carries avoidance metadata when the separation rule fires |
| missiondt/agents/<id>/register | yes | Agent announces domain (aerial/surface) and kind (physical/virtual); retained so late subscribers see it |
| missiondt/mission/checkpoints | yes | Named waypoints (P01…) for the 3D view labels |
| missiondt/mission/routes | yes | Planned route per agent, drawn as green lines in the 3D view |

## 2.3 Independence of components

Every consumer — the 3D view, the terminal panel, and any future tool — is an independent MQTT client. They can be started, stopped, or replicated on any machine that reaches the broker without affecting the twin's real-time loop. The same property makes physical and virtual agents interchangeable: relocating a virtual agent to another process or machine requires no code change.

# 3. Installation

Requirements: Python 3.10 or newer, the Mosquitto MQTT broker, and — only on the machine that renders the 3D view — a display and GPU.

## 3.1 Broker (Mosquitto)

- Windows 10/11: installer from mosquitto.org; add 'listener 1883 127.0.0.1' and 'allow_anonymous true' to mosquitto.conf; start the Mosquitto Broker service (services.msc) or run mosquitto.exe manually.
- macOS: brew install mosquitto; then brew services start mosquitto, or run 'mosquitto -v' in a terminal (verbose mode also shows live traffic — a useful behaviour monitor).
- Linux: apt install mosquitto; ensure a localhost listener with anonymous access.

Sanity check: 'mosquitto_sub -h 127.0.0.1 -t t' in one terminal, 'mosquitto_pub -h 127.0.0.1 -t t -m ok' in another; 'ok' must appear in the first.

## 3.2 Python environment

```
git clone https://github.com/ProfessorFilipo/mission-dt.git && cd mission-dt
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install ursina imageio imageio-ffmpeg   # 3D view + video (GPU machine)
pip install rich                            # optional: pretty panel
```

Note (macOS): Ursina 8.x requires Python 3.11+; on Python 3.10 pip installs Ursina 7.0, which this application also supports.

# 4. Running a mission

A full session uses up to four terminals, all with the virtual environment active:

```
mosquitto -v                          # 1 — broker (or system service)
python experiments/demo_mission.py    # 2 — mission core + virtual agents
python viz/mission_viz.py             # 3 — 3D mission view
python experiments/panel.py           # 4 — live terminal dashboard
```

The mission runs headless: terminal 2 only prints one status line. All visual behaviour lives in the 3D view, which may be opened or closed at any time.

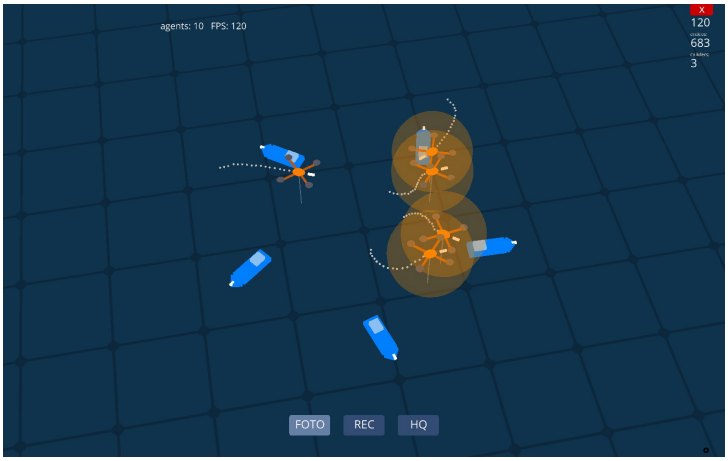


*Figure 2 — The 3D mission view during a separation conflict: orange safety spheres indicate agents inside the 12 m threshold while corrective actuation is being issued.*

## 4.1 3D view controls

| Button | Key | Action |
|---|---|---|
| FOTO | P | Screenshot of the scene, saved to captures/ |
| REC / STOP | R | Start/stop video recording; on stop, frames are encoded to an H.264 MP4 (YouTube/Vimeo-compatible) in captures/ |
| HQ / LQ | H | Toggle recording quality: 30 fps high / 15 fps low |
| ID | I | Toggle drone name labels above each vehicle |
| ROTA | K | Toggle planned-route lines (green); grey dots remain the travelled trail |
| LEG | L | Toggle the translucent legend describing every icon and colour |
| PANEL | O | Toggle a floating, translucent agent table inside the 3D window |
| — | G | Toggle the sky grid |
| — | ESC | Quit (stops any active recording first) |

Camera: right-drag to orbit, scroll to zoom, middle-drag to pan.

## 4.2 Status visualization

- Communication loss: an agent with no telemetry for 1.5 s turns grey; its trail pauses.
- Low battery: a pulsing red marker appears above the vehicle when battery voltage drops below 17.6 V.
- Safety spheres (12 m true-scale radius): hidden when clear; faint white when a same-domain neighbour is within 24 m; orange inside the 12 m separation threshold; red under 3 m (near-collision).

## 4.3 The agent panel

The panel (terminal or floating) shows one row per agent: identifier, domain, kind, position, altitude, speed, battery, telemetry rate, message age, and status (OK, STALE, or LOWBATT). It is the primary debugging tool: killing the mission process, for instance, turns every row STALE within two seconds — the same event that greys the vehicles in the 3D view.

# 5. Configuration reference

Missions are described by a JSON file passed with --config (default: configs/mission_default.json).

```
{ "profile": "orbitas",
  "drones": { "aerial": 5, "surface": 5 },
  "radius_m": 45.0, "arrive_m": 3.0, "aerial_alt_m": 15.0,
  "checkpoints": {}, "assignments": {} }
```

| Field | Meaning |
|---|---|
| profile | Route generator when no explicit assignments exist: orbitas, paralelo, explorador, aleatorio |
| drones | Fleet composition; any number of aerial and surface agents |
| radius_m | Radius of the deployment circle (and scale of generated routes) |
| arrive_m | Distance at which a waypoint counts as reached |
| aerial_alt_m | Cruise altitude assigned to aerial agents |
| checkpoints | Named waypoints: "P01": [lat, lon, alt_m]; alt > 0 is an air corridor, 0 is the surface, < 0 is submerged |
| assignments | Explicit route per drone as an ordered checkpoint list; checkpoints may be shared by several drones |

## 5.1 Route profiles

- orbitas — agents start on a circle and patrol to the antipodal point and back, forcing continuous crossings at the centre; the richest scenario for observing swarm separation.
- paralelo — parallel lanes with back-and-forth transit; useful for formation-style coverage.
- explorador — surface vessels advance along lanes while each aerial drone continuously scouts a moving point 25 m ahead of its paired vessel (a dynamic goal recomputed four times per second); the basis for the planned obstacle-mapping capability.
- aleatorio — every agent visits three random waypoints and then converges on a destination shared by the whole fleet; routes are regenerated at each lap.

## 5.2 Checkpoints in the 3D view

Checkpoints are published retained on MQTT, so the 3D view labels them (P01, P02, …) with no configuration file of its own. Colour encodes the medium: cyan for surface, orange for air corridors, and violet for submerged points, whose label also shows the depth (e.g., "P03 (-2m)"). Planned routes appear as green lines when ROTA is enabled, deliberately distinct from the grey travelled trail so overlapping shared routes remain readable.

# 6. Ready-to-run test scenarios

Each scenario assumes the broker is running and lists the exact command, what to enable in the 3D view, and the expected behaviour.

## Scenario T1 — Orbitas (default): swarm separation

```
python experiments/demo_mission.py
```

- Enable: LEG. Expected: 10 agents cross the centre; safety spheres cycle white → orange near the middle; neighbours veer apart; trails record the avoidance curves.

## Scenario T2 — Paralelo: formation transit

```
# set "profile": "paralelo" in configs/mission_default.json
python experiments/demo_mission.py
```

- Enable: ROTA. Expected: agents shuttle along parallel green lanes 12 m apart; spheres stay mostly white/hidden (lanes match the separation threshold).

## Scenario T3 — Explorador: aerial scouts

```
# "profile": "explorador"
python experiments/demo_mission.py
```

- Enable: ID. Expected: each aerial drone holds position ahead of its paired vessel and turns when the vessel turns; aerial agents show no green route (their goal is dynamic by design).

## Scenario T4 — Aleatorio: converging random routes

```
# "profile": "aleatorio"
python experiments/demo_mission.py
```

- Enable: ROTA. Expected: distinct random green routes that all end at one shared destination; at each lap the lines redraw with fresh routes.

## Scenario T5 — Explicit shared checkpoints

```
python experiments/demo_mission.py --config configs/mission_checkpoints_example.json
```

- Enable: ROTA + ID + LEG. Expected: five labelled pins — two orange air-corridor points at 20 m, two cyan surface points, one violet submerged point — with routes shared by several drones overlapping at P01.

## Scenario T6 — Communication loss

- With any mission and the 3D view running, terminate the mission process (Ctrl+C in its terminal). Expected: within 1.5 s every vehicle turns grey and the panel rows switch to STALE. Restart the mission: agents recolour and resume.

## Scenario T7 — Low battery

- Edit LOW_BATT_V in viz/mission_viz.py from 17.6 to 18.4 and rerun the view. Expected: pulsing red markers above the vehicles within a few minutes as the simulated batteries drain.

## Scenario T8 — Recording a video

- During any scenario press REC, wait ~20 s, press STOP. Expected: console shows frame encoding; an MP4 appears in captures/, playable and ready for YouTube/Vimeo upload.

## Scenario T9 — Reproducing the paper experiments

```
python experiments/run_experiments.py all   # E1 + E2, about 6 minutes
python experiments/run_e3.py                # E3 swarm propagation
python experiments/make_figures.py          # regenerates all figures
```

- Expected: zero frame overruns in E1; roughly 6x uplink reduction in E2; swarm propagation latencies with a median near one frame in E3. Absolute values vary with hardware; the qualitative conclusions should not.

## Scenario T10 — Twin fidelity under packet loss (E4)

```
python experiments/run_e4.py            # loss levels 0%, 5%, 10%
python experiments/run_e4.py 0.2        # or any custom loss probability
```

- Virtual agents accept a loss parameter that drops regulated telemetry with the given probability, emulating a lossy maritime Wi-Fi link. The script compares the twin's per-frame estimate against each agent's 50 Hz ground truth.
- Expected: position RMSE near 1 m and heading RMSE of a few degrees, essentially flat across loss levels; the worst dead-reckoned drift grows with the loss rate (longer stale streaks), bounded by k × 125 ms × vehicle speed for k consecutively lost updates.

## 7. Measured behaviour (summary of experiments)

All figures below come from raw measurements shipped in results/. Reference platform: one virtual CPU (Intel Xeon @ 2.10 GHz, 4 GiB RAM); results were independently reproduced on Windows 10 (i7-9700) and macOS (Apple Silicon).

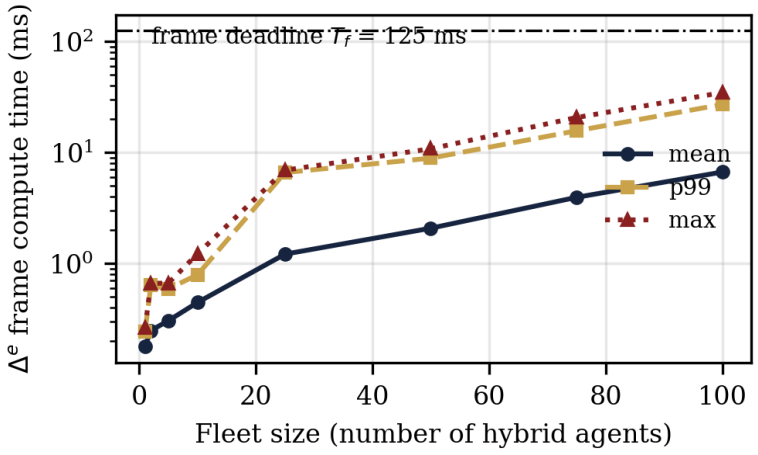


*Figure 3 — E1: frame compute time vs. fleet size. The 125 ms deadline is never missed up to 100 agents.*

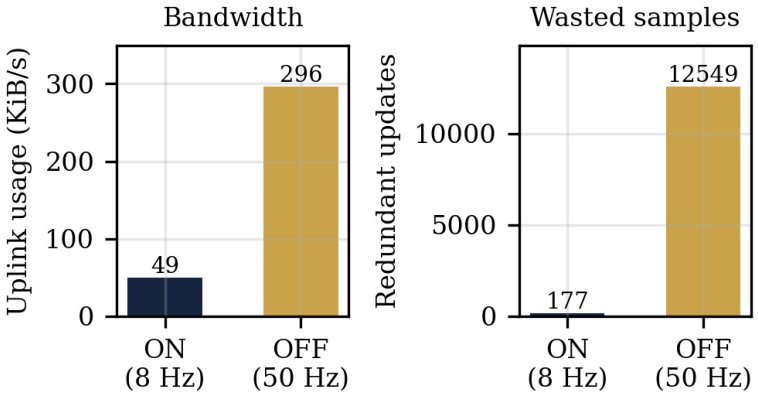


*Figure 4 — E2: bandwidth regulators cut uplink usage ~6x and redundant same-frame samples ~75x.*

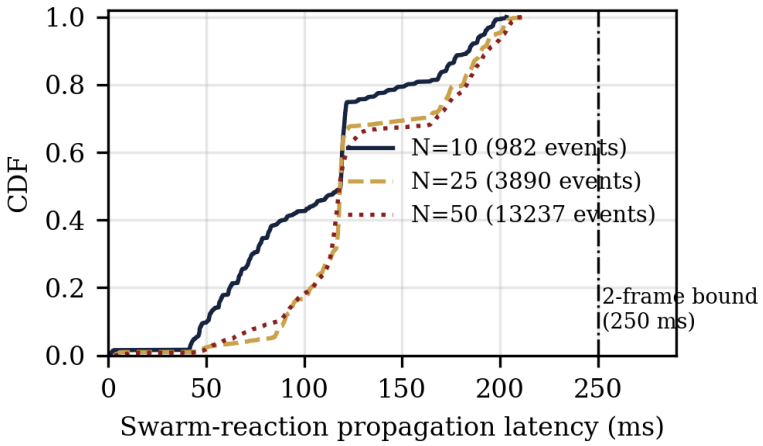


*Figure 5 — E3: swarm-reaction propagation latency (CDF). Median ≈ one frame; every event under the two-frame bound (250 ms).*

Twin fidelity (experiment E4): comparing the twin's estimates against agent ground truth, position RMSE stays between 0.88 m and 0.92 m and heading RMSE at or below 1.7 degrees under 0%, 5%, and 10% injected packet loss; the worst dead-reckoned drift during stale streaks grows from 1.98 m (no loss) to 7.27 m (10% loss), matching the kinematic bound for consecutively lost updates.

Rendering performance of the 3D view: 120 FPS on an Apple-Silicon MacBook Pro and about 50 FPS on a 2010-era GeForce GT 430, with 10 agents, trails, and safety spheres active.

# 8. Troubleshooting

| Symptom | Cause and fix |
|---|---|
| ConnectionRefusedError (errno 61/111) on start | No broker listening on 127.0.0.1:1883. Start Mosquitto; on Mosquitto 2.x add a localhost listener with allow_anonymous true. |
| Agents render but never move | The mission core is not running (or was interrupted). Note: Ctrl+C — including an attempt to copy text in a Windows console — kills the process. |
| 'missing model: cone' warnings / invisible aerial drones | Older visualizer version; update viz/mission_viz.py (procedural quadcopter models do not depend on built-in meshes). |
| Buttons or HUD not visible | Older visualizer with UI placed for ultrawide aspect ratios; current version anchors all UI in the aspect-safe centre region and uses a normal window. |
| brew services start mosquitto fails (Bootstrap failed: 5) | Stale launchd plist. Run the broker manually with 'mosquitto -v', or bootout and remove ~/Library/LaunchAgents/homebrew.mxcl.mosquitto.plist and retry. |
| Video encoding does nothing on STOP | imageio/imageio-ffmpeg not installed; frames are kept as JPGs in captures/ with the exact ffmpeg command printed to the console. |
| Checkpoints not visible | The default configuration has no checkpoints (profiles generate unlabelled routes). Run the checkpoints example config, or verify the retained message with: mosquitto_sub -t missiondt/mission/checkpoints -C 1 |

# 9. Connecting physical vehicles

A physical agent replaces a virtual one by honouring the same MQTT contract. On the vehicle's companion computer (e.g., Raspberry Pi 4 with ArduPilot), run a MAVLink-to-MQTT bridge that publishes ATTITUDE, GLOBAL_POSITION_INT, and SYS_STATUS as the telemetry JSON, and converts actuation messages into RC_OVERRIDE or guided-mode commands. The vehicle's local broker bridges to the ground station over Wi-Fi. From the mission's point of view nothing changes — which is precisely the point.